

M e m e n t o

Memento Pattern

Capture and restore state

DEFINITION

Capture an object's internal state without violating encapsulation, so it can be restored later.

Memento externalizes a snapshot of an object's internal state so it can be rewound later, without exposing those internals. The Originator creates and restores mementos; the Caretaker just holds them as opaque tokens it can't read. The trick is two interfaces — wide for the originator, narrow for everyone else.

WHY IT MATTERS

Undo and rollback need a past state, but exposing the object's fields to save them breaks encapsulation and couples the history-keeper to the internals. A memento lets the originator pack its own state into a token only it can unpack, so the caretaker stacks snapshots without ever seeing inside. The cost is memory — full snapshots of large state add up.

— How it works

Originator	Creates a memento of its state and restores itself from one.
Memento	The snapshot — wide interface for the originator, narrow (opaque) for the caretaker.
Caretaker	Keeps mementos (the history); never reads or mutates their contents.

HOW TO APPLY

1. Give the originator `save()` returning a memento and `restore(m)` to roll back.
2. Pack the state deeply enough that later edits can't reach into the snapshot.
3. Let a caretaker stack mementos for undo; it treats them as opaque tokens.
4. Bound the history (cap, diff, or copy-on-write) so memory doesn't grow forever.

LITMUS TEST

Do you need to restore a past state without exposing the object's internals? If a snapshot must be saved but its contents stay private, that's a Memento.

— Smell → fix

To undo, the caretaker reaches into the editor's private fields:

```
// history reads the editor's internals
snap = { text: editor.text, sel: editor.sel }
// editor.text/sel must be public → leaks state
editor.text = snap.text // anyone can corrupt it
```

↓ LET THE OBJECT SNAPSHOT ITSELF

```
class Editor {
  #text = ''
  type(s: string) { this.#text += s }
  save() { return { text: this.#text } } // opaque token
  restore(m) { this.#text = m.text }
}
const history = []
history.push(editor.save()) // caretaker just stacks it
```

The editor packs and unpacks its own state; the history just stacks the tokens it gets back, never reading inside. Deep-copy mutable state so a later edit can't reach into a saved snapshot, and cap the history to bound memory.

USE WHEN

- ✓ Undo/redo
- ✓ Snapshots, checkpoints, transaction rollback

AVOID WHEN

- ▲ Large, expensive state snapshotted frequently

— Trade-offs & pitfalls

PROS

- + Encapsulation preserved
- + Clean undo

CONS

- Memory cost of snapshots
- Snapshot-frequency tuning

COMMON PITFALLS

- ▲ Memory pressure: full snapshots of large state cost real memory — prefer diffs or copy-on-write as histories grow.
- ▲ A shallow snapshot still shares mutable references, so later edits silently corrupt the past — copy deeply.
- ▲ An unbounded undo stack leaks memory; cap it or age old mementos out (and the caretaker must never mutate one).

WHERE YOU SEE IT

- Editor Ctrl+Z undo/redo; Redux DevTools time-travel stepping through state snapshots.
- SQL SAVEPOINT (partial rollback) and Git commits (each a restorable tree snapshot).
- VM snapshots and game save slots.

SEE ALSO

Command	pairs with Memento for undo stacks — Command records the action, Memento the resulting state
Prototype	clones a copy of state where a deep copy is an acceptable snapshot
State	both deal in object state — State swaps behavior, Memento freezes a restorable copy
Iterator	a robust cursor can snapshot its position as a memento

— Interview & sources

How does it preserve encapsulation?

The memento exposes a wide interface only to the originator; the caretaker holds an opaque token it can't read or change.

Memento vs Command for undo?

Memento snapshots the resulting state; Command records the operation. Command supports redo and is lighter; Memento is simpler for complex state.

Deep or shallow copy?

Deep enough that later mutations can't reach into the saved state — otherwise restore brings back already-corrupted data.

How do you bound memory?

Cap the history, store diffs instead of full snapshots, use copy-on-write, or persist older mementos out of memory.

REMEMBER

Snapshot an object's private state into an opaque token, restore from it later — encapsulation intact.

SOURCES

GoF — "Design Patterns" (1994): Memento (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Memento (leftover patterns)